MARKET INTELLIGENCE FOR USED-CAR EXPORT

Integrated Analysis of the European
Market and U.S. Consumer Preferences

By: N. Bachiorri - D.F. Caramia - H. Mottaran

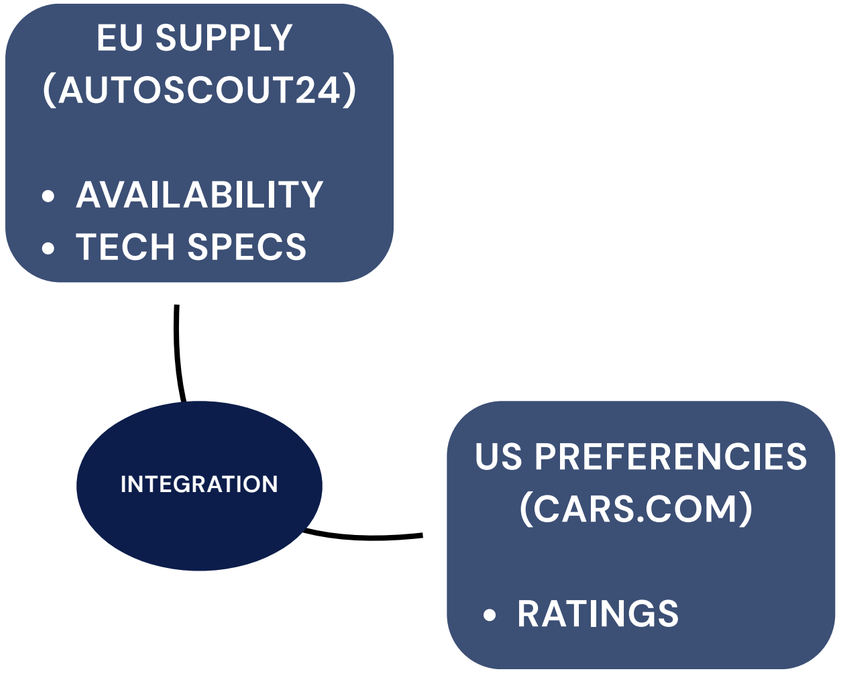
Introduction

- EU market: diverse vehicles with varied technical specifications
- Challenge: Which models/configurations resonate with US buyers?

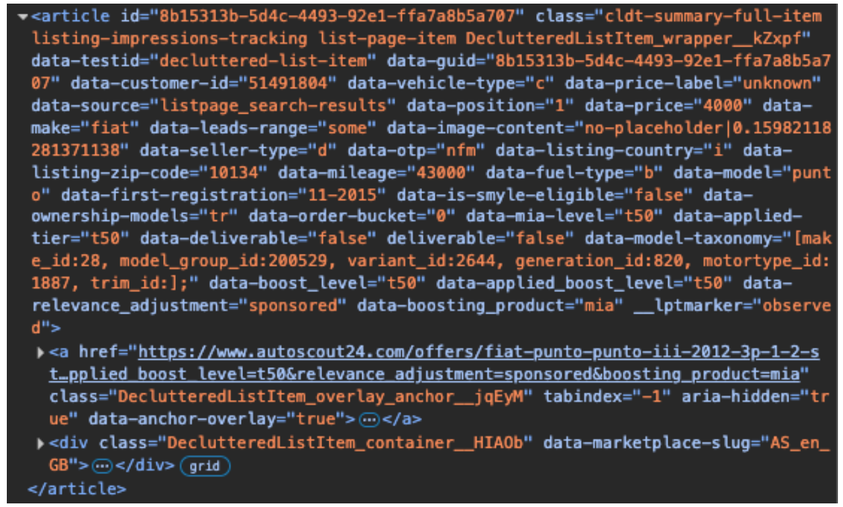
RQ1: Market matching: Which EU models exist in US market?

RQ2: Preference drivers: Which specs → higher US ratings?

RQ3: Decision support: How to rank EU listings by US appeal?



Listings Collection



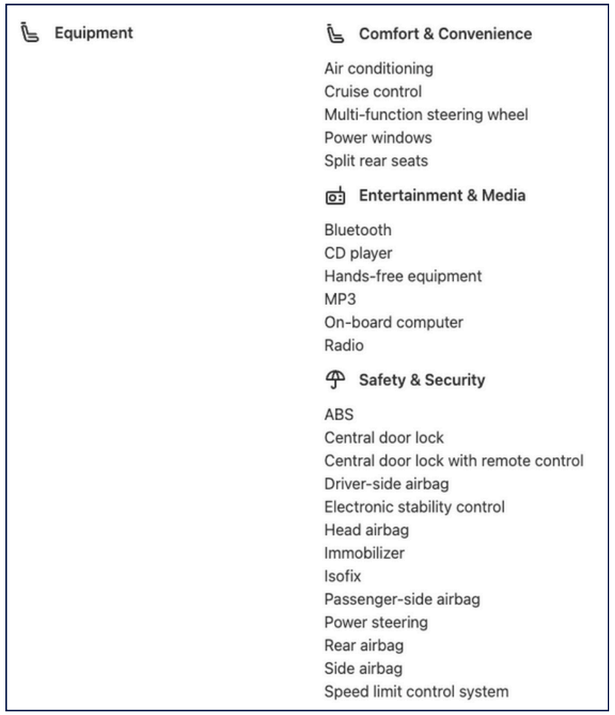
- Data source: AutoScout24.com
- Critical data is present directly in the HTML as data_* attributes → scraping through BeautifulSoup4
- GUID is essential because it provides referential integrity
- Each listing is stored as a JSON document with several important fields (id, price, make, model, mileage, year, fuel. type, link, etc...)
- The <a> tags in the HTML are overwritten by JavaScript at runtime, so we built listings' URLs manually:

  link = f"https://www.autoscout24.com/offers/{make_clean}-{model_clean}-{guid}"

- Output at this point: autoscout24_listings.json (3,633 listing records with base attributes)
- But these are just the surface-level attributes. For technical specifications, we need the detail pages.

Spec & Feat Extraction

- Data source: specific listing's URL
- The challenge here is schema discovery: we didn't know a priori what specifications would be available or how they'd be label. So we manually analyzed ten representative listings to identify common fields.
- However, the most challenging part was features extraction, each vehicle has a different subset of features → we adopted a list of dictionaries for each vehicle instead of boolean columns.
- Output at this point: autoscout24_car_details.json (3,532 listing records with specific attributes)



Embedding the two listings collections

- Merging listings anCCCd details into a single analytics ready dataset requires three transformations:

1. **Flatten Specifications**: our specifications are stored as nested dictionaries. We used pd.json_normalize() to flatten the nested dict into separate columns.
2. **Binary Encoding for Features**: the result is 144 binary columns, this is a sparse matrix but it's necessary for ML model then.
3. **Inner Join**: we keep only vehicles that appear in both sources. The join is performed on the id column (the GUID). We have 3,633 listings but only 3,532 successfully scraped details. We want only complete records for analysis.

- Output at this point: autoscout24_combined_data.json

U.S. Consumer data

Source of data: **Cars.com**
(american platform specialized in car buying and selling, widely used by consumer to evaluate vehicles)

To address the project objectives the main focus is:

- Acquiring data that could be useful
- Create an integrated Dataset (for **cross-market analysis**)

To perform the cross-market Analysis we acquired:

- **Average ratings provided by U.S. users**
- **Number of users who rated each vehiacle**

The rating consiedred were:
Comfort, Interior, Performance, Value, Exterior, Reliability

Data Acquisition Strategy

From a technical perspective, the data acquisition process could have been based on a **REST API**, but was discared because:

- **Long activation times**
- **Charged a fee**

So was adopted **web screaping** (**selenium**: beacuse the website uses **javascript** to render certain page elements such as the **shadow dom**).

To reduce the risk of detection:

- **Random sleep intervals ranging from 30 to 40 seconds**

URL Normalization Strategy

To enable access to the pages of individual vehicles was developed:

- N**ormalization Function**

Worked on:

- Standardize **brand** and **model** making there compatible with url insertion

How?

- **Convert all text to lowercase**
- **Replace spaces and special characters with underscores**
- **Generated uniform slugs**

Weighted Rating Calculation

To avoid downloading an excessive amount of data, it was taken into account that each car model is distributed accross mutiple production years.

Strategy adopted:
Starting from the most recent year and proceeding backwords, year by year, progressively summing the reviews until reachin at least **50 cumulative reviews.**

During this process were acquired, for each year:

- **Average rating**
- **Number of users who rated the model**

Number of users was used to compute:

- **Overall Weighted Rating**

Checkpoint Saving Mechanism

To prevent data loss in case of unexpected issues during execution were introduced:

- **Checkpoint saving mechanism**

Mechanism:

- At the end of the processing of each vehiacle, the collected data were saved into a **json file**
- When moving on to the next vehiacle, the **json file** was **recreated** by including both the data of the new vehiacle and those of the previously processed ones

At the end of execution:

- If the process completed succesfully the entire final dataset was saved into a single json file containing, for each vehiacle **(brand, model, weighted rating, numer of users who provided a rating)**

Data Integration (Merging): Goal & Join Key

Goal: enrich EU used-car listings (AutoScout24) with U.S. preference signals (Cars.com ratings) to support export selection.

Integration key: (make_norm, model_norm) on both sources.

Why normalization: removes formatting noise (case, spaces, special chars) → deterministic and reproducible matching.

Join type: INNER JOIN

- We keep only EU listings for which a U.S. preference signal exists (otherwise no target for ML).
- Conceptual integrity: enforces a **logical referential integrity** — every integrated listing references an existing (make_norm, model_norm) on the Cars.com side.

Data Integration: Output + Integrity Risks

Integrated dataset output: combined_listings_reviews.csv

- **Rows: 1507**
- **Columns: 174**
- **Distinct make/model pairs (used for modeling): 239**

Integrity risks after merge (important):

- If (make_norm, model_norm) is not unique on Cars.com, the join can become many-to-many → row multiplication.
- This produces duplicated listing IDs in the integrated dataset.

Observed issue:

- Uniqueness (id): 99.8% → 3 duplicated id values detected (later removed in ML preparation).

Data Cleansing: Standardization & Type Conversion

After merging, we “clean” the integrated table to make values consistent and usable for queries + ML.

Missing values handling:

- **Replace placeholder values like "-" with NA.**

Parsing / type conversion (unit cleaning):

- year: extract the year value (split / parse)
- spec_empty_weight: remove "kg" + formatting → numeric
- power: extract kW from strings like "135kW(184hp)" → numeric (coerce errors to NA)
- spec_engine_size: remove "cc" + formatting → numeric
- spec_fuel_consumption: extract numeric token → numeric

Why: enforces domain validity (quantities become real quantities, not strings).

Data Cleansing: Feature Engineering for ML Readiness

Categorical → One-hot encoding (dummies):

seller_type, spec_drive, spec_paint_type, spec_fuel_type, spec_transmission,

spec_color, spec_body_type, spec_emission_class, spec_vehicle_type

Binary features:

- Identify feat_* columns and enforce boolean logic (0/1 → boolean)

Output artifact: combined_listings_reviews_dummified.csv

SQLite DB Structure

DB choice: SQLite.

Database file: my_project.db

Table: cars_data

- **Rows: 1507**
- **Columns: 174**

Table content (logical groups):

- Listing metadata: id, make, model, year, price, mileage, ...
- AutoScout24 technical specs: spec_*
- AutoScout24 equipment/features: feat_*
- Cars.com preference signals: rating dimensions + cumulata

Example SQL Queries

We used SQL for quick sanity checks and exploration.

Query example 1: models with low consumption

```sql
SELECT DISTINCT
    make,
    model
FROM cars_data
WHERE spec_fuel_consumption < 7.0;
```

Index	Make	Model
0	bmw	320
1	ford	focus
2	ford	fiesta
3	fiat	500
4	jaguar	e-pace
5	audi	a3
6	jaguar	xe
7	volkswagen	tiguan
8	bmw	x3
9	toyota	yaris
10	volkswagen	golf
11	bmw	x1
12	jaguar	f-pace
13	bmw	318
14	audi	a4
15	audi	q3
16	land rover	range rover evoque
17	volvo	v90
18	bmw	530
19	audi	a4 allroad
20	land rover	range rover sport

Query example 2: average price by make

```sql
SELECT
    make,
    AVG(price)
FROM cars_data
GROUP BY make
ORDER BY AVG(price) DESC;
```

Index	Make	AVG(price)
0	rolls-royce	399640.00
1	lamborghini	276130.00
2	ferrari	214830.00
3	mclaren	149570.00
4	aston martin	99999.00
5	ram	89500.00
6	maserati	75797.14
7	bentley	72983.33
8	chrysler	72750.00
9	porsche	66893.70
10	mercedes-benz	53997.24
11	land rover	47555.70
12	bmw	42855.08
13	dodge	41119.67
14	audi	32548.89
15	lexus	30566.17
16	tesla	29548.59
17	lotus	28900.00
18	alfa romeo	28543.68
19	volvo	25812.61
20	polestar	24946.67

Data Quality Assessment (Measured Values)

We measured two core dimensions after integration:

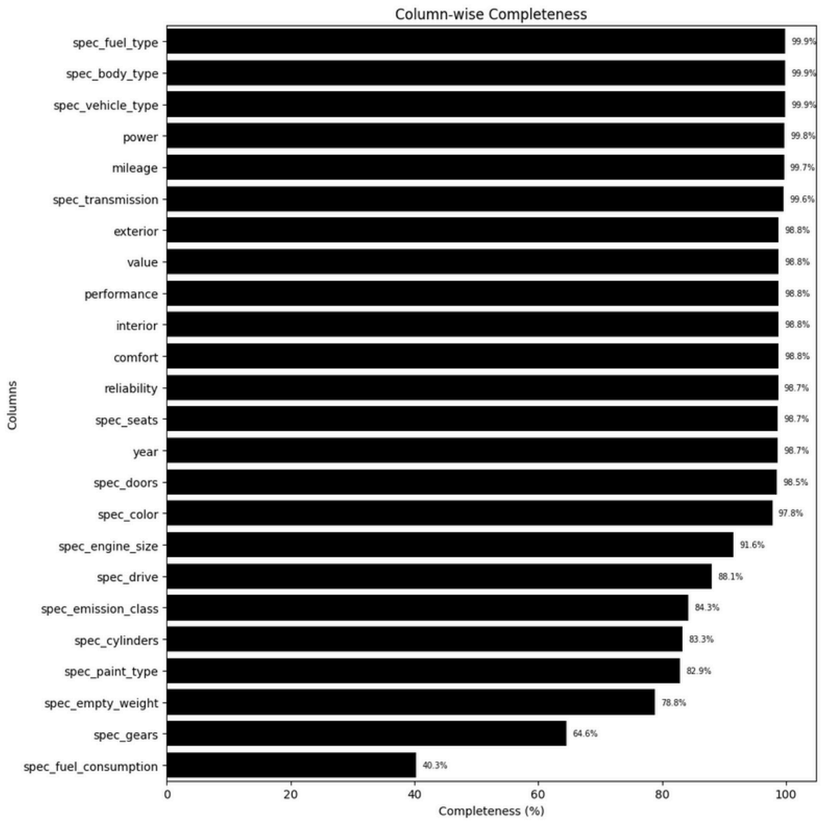
Completeness (non-missing / total cells): 98.85%

- computed on the integrated dataset

Uniqueness (distinct id / rows): 99.8%

- **3 duplicated listing IDs** detected
- duplicates are consistent with merge risk (non-unique join key on Cars.com side)

Action: duplicates removed in the ML preparation step.



Target Engineering: Weighted PCA → “appreciate”

Cars.com provides six rating dimensions:

comfort, interior, performance, value, exterior, reliability

Handling missing reliability:

- fill reliability using the row mean of the other rating dimensions.

Weighted PCA:

- weights = cumulata (review volume) → models with more reviews influence the signal more.
- take **PC1** as a single “preference score”.

Binary target definition:

- **appreciate = (PC1 > mean(PC1))**

Final ML dataset size: **1486 rows**

Class distribution: **936 False, 550 True**

Output artifact: combined_listings_reviews_final.csv

ML Model: Training & Tuning (Gradient Boosting)

Model: **Gradient Boosting Classifier**

Split: train/test = 80/20, stratified, random state 42

Tuning: GridSearchCV, 5-fold CV

Optimization metric: ROC-AUC

Cross-validated ROC-AUC (tuned): 0.8624

Model Performance (Test Set): Baseline vs Tuned

Baseline model (test):

- Accuracy 0.8020
- Precision 0.7576
- Recall 0.6818
- F1 0.7177
- ROC-AUC 0.8571

Tuned model (test):

- Accuracy 0.8322
- Precision 0.7830
- Recall 0.7545
- F1 0.7685
- ROC-AUC 0.8838

Interpretability: Feature Importance + LIME (Global Summary)

We used two explanation layers:

1. **Native feature importance** (from Gradient Boosting)
2. **LIME** explanations aggregated to identify global drivers + **directionality**

Key drivers (consistently):

- spec_body_type_Off-Road/Pick-up
- power
- spec_engine_size

Directional patterns from LIME:

- Strong positive: spec_body_type_Convertible
- Positive: higher power, higher spec_engine_size
- Strong negative: spec_fuel_type_Electric
- Negative: van-like body types and features such as right-hand drive

Why interpretability matters: **supports justification of export candidates, not just prediction.**

Conclusions + Limitations + Next Steps

We connected EU supply (AutoScout24 listings) with U.S. preferences (Cars.com ratings) and built a pipeline that produces an appreciation label + interpretable drivers.

Best model performance: ROC-AUC 0.8838 (test)

Main limitations:

- Regulatory feasibility and export costs not modeled
- EU data is a single snapshot (no temporal dynamics)
- Matching limited to make/model (no trim/version-level resolution)

Future work:

- multi-snapshot collection (timeliness trends)
- stronger entity resolution (beyond slug)
- integrate feasibility constraints for a full export decision model